torch.func.vmap#

https://pytorch.org/docs/stable/generated/torch.func.vmap.html

Description:

vmap is the vectorizing map; vmap(func) returns a new function that maps func over some dimension of the inputs. Semantically, vmap pushes the map into PyTorch operations called by func, effectively vectorizing those operations. vmap is useful for handling batch dimensions: one can write a function func that runs on examples and then lift it to a function that can take batches of examples with vmap(func). vmap can also be used to compute batched gradients when composed with autograd. torch.vmap() is aliased to torch.func.vmap() for convenience. Use whichever one you'd like. func (function) – A Python function that takes one or more arguments. Must return one or more Tensors.

Function Signature

```
torch.func.vmap(func, in_dims=0, out_dims=0,
randomness='error', *, chunk_size=None)[source]#

Parameters

Returns

Return type
```

Returns:

Callable

Code Examples

```
>>> torch.dot  # [D], [D] -> []
>>> batched_dot = torch.func.vmap(torch.dot)  # [N, D], [N, D] -
> [N]
>>> x, y = torch.randn(2, 5), torch.randn(2, 5)
>>> batched_dot(x, y)
```

```
>>> batch_size, feature_size = 3, 5
>>> weights = torch.randn(feature_size, requires_grad=True)
>>>
>>> def model(feature_vec):
>>> # Very simple linear model with activation
>>>     return feature_vec.dot(weights).relu()
>>>
>>> examples = torch.randn(batch_size, feature_size)
>>> result = torch.vmap(model)(examples)
```

```
>>> # Setup
>>> N = 5
>>> f = lambda x: x**2
>>> x = torch.randn(N, requires_grad=True)
>>> y = f(x)
>>> I_N = torch.eye(N)
>>>
>>> # Sequential approach
>>> jacobian_rows = [torch.autograd.grad(y, x, v,
retain_graph=True)[0]
>>>                  for v in I_N.unbind()]
>>> jacobian = torch.stack(jacobian_rows)
>>>
>>> # vectorized gradient computation
>>> def get_vjp(v):
>>>     return torch.autograd.grad(y, x, v)
>>> jacobian = torch.vmap(get_vjp)(I_N)
```

```
>>> torch.dot  # [D], [D] -> []
>>> batched_dot = torch.vmap(
...     torch.vmap(torch.dot)
... )  # [N1, N0, D], [N1, N0, D] -> [N1, N0]
>>> x, y = torch.randn(2, 3, 5), torch.randn(2, 3, 5)
>>> batched_dot(x, y)  # tensor of size [2, 3]
```

```
>>> torch.dot  # [N], [N] -> []
>>> batched_dot = torch.vmap(torch.dot, in_dims=1)  # [N, D],
[N, D] -> [D]
>>> x, y = torch.randn(2, 5), torch.randn(2, 5)
>>> batched_dot(
...     x, y
... )  # output is [5] instead of [2] if batched along the 0th
dimension
```

```
>>> torch.dot  # [D], [D] -> []
>>> batched_dot = torch.vmap(torch.dot, in_dims=(0, None))  #
[N, D], [D] -> [N]
>>> x, y = torch.randn(2, 5), torch.randn(5)
>>> batched_dot(
...     x, y
... )  # second arg doesn't have a batch dim because in_dim[1]
was None
```

```
>>> f = lambda dict: torch.dot(dict["x"], dict["y"])
>>> x, y = torch.randn(2, 5), torch.randn(5)
>>> input = {"x": x, "y": y}
>>> batched_dot = torch.vmap(f, in_dims=({"x": 0, "y": None},))
>>> batched_dot(input)
```

```
>>> f = lambda x: x**2
>>> x = torch.randn(2, 5)
>>> batched_pow = torch.vmap(f, out_dims=1)
>>> batched_pow(x)  # [5, 2]
```

```
>>> x = torch.randn([2, 5])
>>> def fn(x, scale=4.):
>>>   return x * scale
>>>
>>> batched_pow = torch.vmap(fn)
>>> assert torch.allclose(batched_pow(x), x * 4)
>>> batched_pow(x, scale=x)  # scale is not batched, output has
shape [2, 2, 5]
```

Notes & Warnings

NOTE Note torch.vmap() is aliased to torch.func.vmap() for convenience. Use whichever one you'd like.

NOTE Note vmap does not provide general autobatching or handle variable-length sequences out of the box.

Detailed Documentation

Documentation

vmap is the vectorizing map; vmap(func) returns a new function that maps func over some dimension of the inputs. Semantically, vmap pushes the map into PyTorch operations called by func, effectively vectorizing those operations.

vmap is useful for handling batch dimensions: one can write a function func that runs on examples and then lift it to a function that can take batches of examples with vmap(func). vmap can also be used to compute batched gradients when composed with autograd.

torch.vmap() is aliased to torch.func.vmap() for convenience. Use whichever one you'd like.

func (function) – A Python function that takes one or more arguments. Must return one

or more Tensors.

in_dims (int or nested structure) – Specifies which dimension of the inputs should be mapped over. in_dims should have a structure like the inputs. If the in_dim for a particular input is None, then that indicates there is no map dimension. Default: 0.

out_dims (int or Tuple[int]) – Specifies where the mapped dimension should appear in the outputs. If out_dims is a Tuple, then it should have one element per output. Default: 0.

randomness (str) – Specifies whether the randomness in this vmap should be the same or different across batches. If 'different', the randomness for each batch will be different. If 'same', the randomness will be the same across batches. If 'error', any calls to random functions will error. Default: 'error'. WARNING: this flag only applies to random PyTorch operations and does not apply to Python's random module or numpy randomness.

chunk_size (None or int) – If None (default), apply a single vmap over inputs. If not None, then compute the vmap chunk_size samples at a time. Note that chunk_size=1 is equivalent to computing the vmap with a for-loop. If you run into memory issues computing the vmap, please try a non-None chunk_size.

Returns a new "batched" function. It takes the same inputs as func, except each input has an extra dimension at the index specified by in_dims. It takes returns the same outputs as func, except each output has an extra dimension at the index specified by out_dims.

One example of using vmap() is to compute batched dot products. PyTorch doesn't provide a batched torch.dot API; instead of unsuccessfully rummaging through docs, use vmap() to construct a new function.

vmap() can be helpful in hiding batch dimensions, leading to a simpler model authoring

experience.

vmap() can also help vectorize computations that were previously difficult or impossible to batch. One example is higher-order gradient computation. The PyTorch autograd engine computes vjps (vector-Jacobian products). Computing a full Jacobian matrix for some function f: R^N -> R^N usually requires N calls to autograd.grad, one per Jacobian row. Using vmap(), we can vectorize the whole computation, computing the Jacobian in a single call to autograd.grad.

vmap() can also be nested, producing an output with multiple batched dimensions

If the inputs are not batched along the first dimension, in_dims specifies the dimension that each inputs are batched along as

If there are multiple inputs each of which is batched along different dimensions, in_dims must be a tuple with the batch dimension for each input as

If the input is a Python struct, in_dims must be a tuple containing a struct matching the shape of the input:

By default, the output is batched along the first dimension. However, it can be batched along any dimension by using out_dims

For any function that uses kwargs, the returned function will not batch the kwargs but will accept kwargs

vmap does not provide general autobatching or handle variable-length sequences out of the box.